학습 교안

오차역전파 (Backpropagation) 이해하기

계산 그래프 · 순전파 · 로컬/글로벌 그래디언트 · 연쇄법칙 · 덧셈/곱셈 노드

항목	내용
원본 영상	테디노트 TeddyNote, 「오차역전파 (Backpropagation)의 개념을 쉽게 이해해 봅시다」 (12분 5초, 2020.11.29)
영상 주소	https://www.youtube.com/watch?v=1Q_etC_GHHk
대상	경사하강법을 배운 뒤, 신경망이 '모든 가중치의 기울기'를 어떻게 구하는지 궁금한 학생
선수 지식	편미분 · 연쇄법칙 (「ML_기초수학_학습교안」 8~9장), 경사하강법 (「경사하강법_기본수식_학습교안」)
학습 시간	영상 시청 12분 + 교안 정리 및 실습 약 2시간
준비물	Python 3, numpy (실습 코드 폴더의 README 참고)

목차

0장 들어가며 . . . 3

1장 오차역전파란? . . . 4

2장 계산 그래프와 순전파 . . . 5

3장 로컬 그래디언트 . . . 7

4장 연쇄법칙으로 ∂f/∂x 구하기 . . . 9

5장 로컬 그래디언트 × 글로벌 그래디언트 . . . 11

6장 덧셈 · 곱셈 노드의 규칙 . . . 13

7장 정리와 코드 구현 . . . 14

8장 뉴런 하나에 적용하기 (보충) . . . 16

부록 A 확인 문제 정답 및 해설 . . . 19

부록 B 핵심 정리 한눈에 보기 . . . 20

CHAPTER 0

들어가며

경사하강법 교안에서 w, b 두 개의 기울기를 손으로 유도했습니다. 그런데 실제 신경망에는 가중치가 수백만 개 있고, 층이 깊게 쌓여 있습니다. 이 모든 가중치의 기울기를 하나하나 손으로 유도할 수는 없겠죠. 이 문제를 해결하는 알고리즘이 **오차역전파(Backpropagation)**입니다.

영상은 아주 작은 식 f = (x + y)z 하나로 핵심을 보여 줍니다. 복잡한 신경망도 결국 **덧셈·곱셈 같은 작은 계산이 이어 붙은 것**이고, 각 계산의 미분을 **연쇄법칙으로 곱해 나가기만 하면** 된다는 것이 요지입니다. 이 교안은 영상 내용을 정리하고, 7·8장에서 역전파를 직접 코드로 구현해 경사하강법 교안의 결과와 똑같이 나오는지 확인합니다.

교안 구성

장	주제	영상 구간	한 줄 요약
1	오차역전파란?	00:00~01:03	오차를 뒤로 전파하며 가중치를 갱신
2	계산 그래프와 순전파	01:03~02:36	f = (x + y)z 를 노드로 그리고 앞으로 계산
3	로컬 그래디언트	02:36~05:36	각 노드에서 자기 입력에 대한 미분
4	연쇄법칙으로 ∂f/∂x 구하기	05:36~07:43	∂f/∂q × ∂q/∂x = -4
5	로컬 × 글로벌 그래디언트	07:43~10:43	뒤에서 내려온 값 × 내 로컬 값
6	덧셈 · 곱셈 노드의 규칙	10:43~11:25	덧셈은 그대로, 곱셈은 자리 바꾸기
7	정리와 코드 구현	11:25~끝	Add · Mul 클래스로 역전파 구현
8	뉴런 하나에 적용하기 (보충)	-	역전파 = 경사하강법 교안의 기울기 공식

공부하는 방법

1) 각 장의 **영상 구간**을 보고, 2) 교안의 그래프 그림에 **직접 숫자를 적어 가며** 따라 하세요. 3) 실습 코드로 같은 숫자가 나오는지 확인하고, 4) 확인 문제를 풀고 부록 A 와 맞춰 봅니다.

핵심 포인트

- 이 교안의 그림 색 규칙: **초록** = 순전파 값, **빨강** = 로컬 그래디언트, **주황** = 역전파로 내려오는 미분값.
- 모든 실습 결과는 실제로 실행한 값입니다.

CHAPTER 1

오차역전파란?

영상 구간 **00:00~01:03** 인공신경망을 학습시키는 알고리즘, 차이를 뒤로 전파해 가중치 갱신

학습 목표
- 오차역전파가 무엇을 위한 알고리즘인지 한 문장으로 설명할 수 있다.
- 순전파와 역전파의 방향을 구분할 수 있다.

오차역전파는 **인공신경망(Artificial Neural Network)을 학습시키는 알고리즘**입니다. 과정은 다음과 같습니다.

단계	하는 일
① 순전파	입력을 넣어 앞쪽(입력)에서 뒤쪽(출력)으로 계산해 모델의 출력값을 얻는다
② 오차 계산	내가 원하는 **타깃 값**과 모델의 **출력값**이 얼마나 차이 나는지(손실) 계산한다
③ 역전파	그 차이를 **뒤에서 앞으로** 전파하면서 각 노드의 가중치가 손실에 주는 영향(미분값)을 구한다
④ 갱신	구한 미분값으로 가중치를 업데이트한다 (경사하강법)

경사하강법 교안에서 배운 업데이트 식 $w \leftarrow w - \eta \cdot \partial L/\partial w$ 를 떠올려 보세요. 경사하강법을 쓰려면 **모든 가중치에 대한 $\partial L/\partial w$** 가 필요합니다. 오차역전파는 바로 이 값들을 **빠르고 기계적으로** 계산하는 방법입니다. 즉 역전파가 기울기를 구하고, 경사하강법이 그 기울기로 이동합니다.

주의
- '오차를 뒤로 전파한다'는 말은 정확히는 **손실의 미분값(그래디언트)**을 뒤로 전달한다는 뜻입니다. 오차 값 자체가 그대로 거꾸로 흐르는 것은 아닙니다.

핵심 포인트
- 역전파 = 모든 가중치의 기울기($\partial L/\partial w$)를 구하는 알고리즘.
- 순전파는 앞 → 뒤로 값 계산, 역전파는 뒤 → 앞으로 미분 계산.

확인 문제

Q1. 오차역전파와 경사하강법은 각각 학습 과정에서 어떤 역할을 하는가?
Q2. 순전파와 역전파는 각각 어느 방향으로 계산이 진행되는가?

CHAPTER 2

계산 그래프와 순전파

영상 구간 **01:03~02:36** f(x, y, z) = (x + y)z, q 로 치환, x = -2, y = 5, z = -4 대입

학습 목표
- 식을 계산 그래프(노드와 선)로 그릴 수 있다.
- 순전파로 중간값과 출력값을 계산할 수 있다.

영상이 사용하는 예제 식입니다. 입력이 x, y, z 세 개입니다.

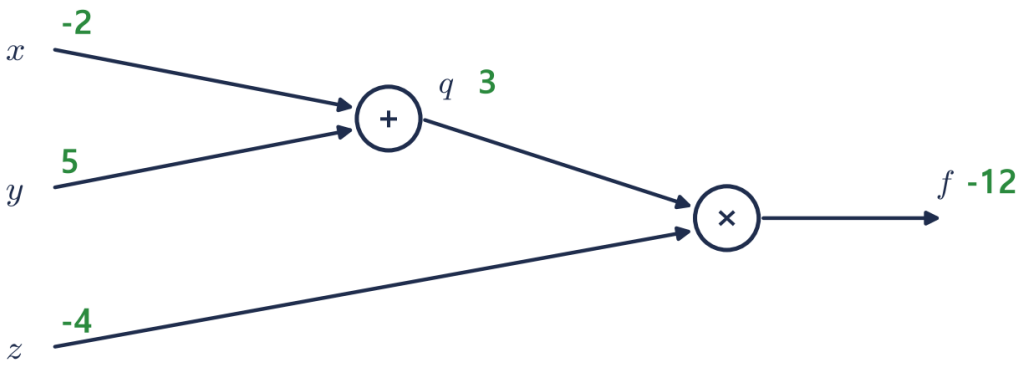
$$f(x, y, z) = (x + y)\,z$$

괄호 안 x + y 를 **q** 로 치환하면 식이 두 개의 작은 계산으로 나뉩니다.

$$q = x + y \qquad\qquad f = q\,z$$

이것을 그림으로 그린 것이 **계산 그래프(computational graph)**입니다. 동그라미(노드)는 하나의 연산이고, 선은 값이 흘러가는 길입니다. x, y 가 **덧셈 노드**로 들어가 q 가 되고, q 와 z 가 **곱셈 노드**로 들어가 f 가 됩니다.

이제 x = -2, y = 5, z = -4 를 넣어 왼쪽부터 계산합니다. q = -2 + 5 = 3, f = 3 × (-4) = -12. 이렇게 **입력에서 출력 방향으로 차례로 계산하는 과정**을 **순전파(Forward Pass)**라고 합니다.



순전파 (Forward Pass) : 왼쪽 → 오른쪽으로 값을 계산

그림 2-1. f = (x + y)z 의 계산 그래프와 순전파 값

실습 코드 2-1 - 순전파

```
x, y, z = -2, 5, -4        # 입력값

# 순전파 (Forward Pass)
q = x + y                  # 덧셈 노드
f = q * z                  # 곱셈 노드
print("q =", q)
print("f =", f)
```

실행 결과

```
q = 3
f = -12
```

신경망도 똑같습니다. 입력 × 가중치, 더하기 편향, 활성화 함수 같은 작은 연산 노드가 수천~수백만 개 연결된 거대한 계산 그래프일 뿐입니다.

핵심 포인트

- 복잡한 식 = 작은 연산 노드들의 연결 (계산 그래프).
- 순전파: 입력 → 출력 방향으로 값을 계산.

확인 문제

Q1. x = 1, y = 2, z = 3 일 때 순전파로 q 와 f 를 구하시오.
Q2. 식 g = (a × b) + c 를 계산 그래프로 그리면 어떤 노드가 몇 개 필요한가?

CHAPTER 3

로컬 그래디언트

영상 구간 **02:36~05:36** ∂q/∂x, ∂q/∂y, ∂f/∂q, ∂f/∂z 를 순전파 때 구해 두기

학습 목표
- 미분값의 의미(입력이 변할 때 출력의 변화량)를 설명할 수 있다.
- 각 노드의 로컬 그래디언트를 구할 수 있다.

3.1 미분값의 의미

∂q/∂x 는 "**x 가 아주 조금 변할 때 q 는 얼마나 변하는가**"를 나타냅니다. 예를 들어 이 값이 2 라면, x 가 1 만큼 변할 때 q 는 약 2 만큼 변한다는 뜻입니다.

3.2 덧셈 노드 q = x + y

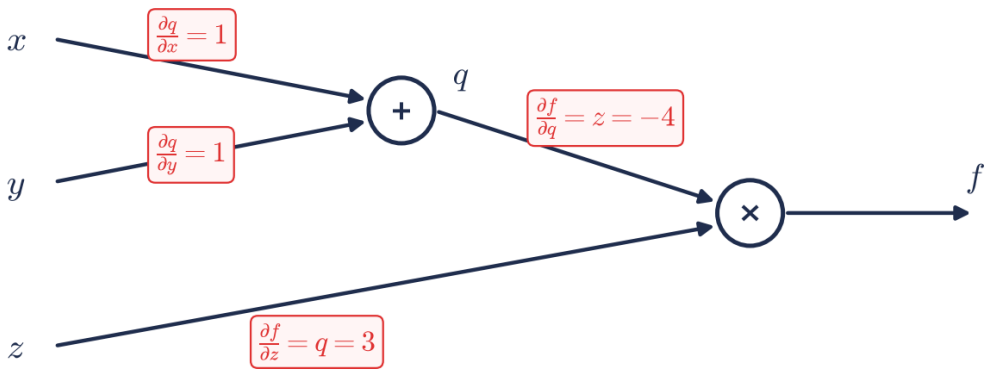
편미분 규칙(나머지 변수는 상수 취급)을 쓰면 간단합니다. x 로 미분하면 y 는 상수라 사라지고 x 는 1 이 됩니다.

$$\frac{\partial q}{\partial x} = 1 \qquad \frac{\partial q}{\partial y} = 1$$

3.3 곱셈 노드 f = q · z

q 로 미분하면 z 가 상수처럼 남고, z 로 미분하면 q 가 남습니다. 순전파에서 z = -4, q = 3 을 이미 알고 있으므로 숫자를 바로 넣을 수 있습니다.

$$\frac{\partial f}{\partial q} = z = -4 \qquad \frac{\partial f}{\partial z} = q = 3$$



로컬 그래디언트 : 각 노드에서 '자기 입력'에 대한 미분 (순전파 때 미리 구해 둠)

그림 3-1. 각 노드에서 구한 로컬 그래디언트

이처럼 **한 노드 안에서만** 계산하는 미분, 즉 노드의 출력을 그 노드의 **직접 입력**으로 미분한 값을 **로컬 그래디언트(local gradient)**라고 합니다. 로컬 그래디언트는 그 노드의 입력값만 있으면 계산되므로 **순전파를 하면서 미리 구해 둘 수 있습니다**.

실습 코드 3-1 - 로컬 그래디언트

```
# 로컬 그래디언트: 각 노드에서 '자기 입력'에 대한 미분
dq_dx = 1                  # q = x + y  ->  ∂q/∂x = 1
dq_dy = 1                  #               ∂q/∂y = 1
df_dq = z                  # f = q * z  ->  ∂f/∂q = z
df_dz = q                  #               ∂f/∂z = q
print("∂q/∂x =", dq_dx, " ∂q/∂y =", dq_dy)
print("∂f/∂q =", df_dq, " ∂f/∂z =", df_dz)
```

실행 결과

```
∂q/∂x = 1  ∂q/∂y = 1
∂f/∂q = -4  ∂f/∂z = 3
```

핵심 포인트

- 로컬 그래디언트 = 노드 출력을 그 노드의 직접 입력으로 미분한 값.
- 순전파 때 입력값을 알게 되므로 그때 미리 구해 저장할 수 있다.

확인 문제

Q1. x = 1, y = 2, z = 3 일 때 네 개의 로컬 그래디언트 ∂q/∂x, ∂q/∂y, ∂f/∂q, ∂f/∂z 를 구하시오.
Q2. f = q · z 에서 ∂f/∂q 가 z 가 되는 이유를 설명하시오.

CHAPTER 4

연쇄법칙으로 ∂f/∂x 구하기

영상 구간 **05:36~07:43** 궁극적으로 구하고 싶은 것은 ∂f/∂x, 체인 룰 사용

학습 목표
- 연쇄법칙으로 ∂f/∂x, ∂f/∂y 를 계산할 수 있다.
- 계산한 미분값을 해석할 수 있다.

우리가 궁극적으로 알고 싶은 것은 "**입력 x 가 변할 때 최종 출력 f 가 얼마나 변하는가**", 즉 ∂f/∂x 입니다. 그런데 x 는 f 에 직접 연결되어 있지 않고 q 를 거쳐 연결되어 있어서 한 번에 구할 수 없습니다. 이때 **연쇄법칙(chain rule)**을 씁니다.

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x} = (-4)(1) = -4$$

분수처럼 보면 가운데 ∂q 가 약분되어 ∂f/∂x 가 남는 모양입니다. 필요한 두 값은 3장에서 이미 구해 두었으므로 **곱하기만** 하면 됩니다. y 도 똑같습니다.

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y} = (-4)(1) = -4 \qquad \frac{\partial f}{\partial z} = q = 3$$

해석

∂f/∂x = -4 는 "x 가 1 증가하면 f 는 -4 만큼 증가, 즉 **4 만큼 감소**한다"는 뜻입니다. 실제로 x = -1 로 바꾸면 q = 4, f = 4 × (-4) = -16 으로 -12 에서 정확히 4 줄어듭니다.

실습 코드 4-1 - 연쇄법칙으로 미분

```
# 연쇄법칙: (뒤에서 내려온 미분) x (로컬 그래디언트)
df_dx = df_dq * dq_dx
df_dy = df_dq * dq_dy
print("∂f/∂x =", df_dx)
print("∂f/∂y =", df_dy)
print("∂f/∂z =", df_dz)
```

실행 결과

```
∂f/∂x = -4
∂f/∂y = -4
∂f/∂z = 3
```

실습 코드 4-2 - 수치 미분으로 검산

```
def F(x, y, z):
    return (x + y) * z

h = 1e-6
for name, d in [("x", (h, 0, 0)), ("y", (0, h, 0)), ("z", (0, 0, h))]:
    plus  = F(x + d[0], y + d[1], z + d[2])
    minus = F(x - d[0], y - d[1], z - d[2])
    print(f"수치 미분 ∂f/∂{name} =", round((plus - minus) / (2 * h), 4))

print("x 를 1 늘리면 f:", F(x, y, z), "->", F(x + 1, y, z))
```

실행 결과

```
수치 미분 ∂f/∂x = -4.0
수치 미분 ∂f/∂y = -4.0
수치 미분 ∂f/∂z = 3.0
x 를 1 늘리면 f: -12 -> -16
```

주의

• 이 예제는 f 가 x 에 대해 1차식이라 'x 가 1 늘면 f 가 정확히 -4' 가 성립합니다. 일반적인 함수에서 미분값은 **아주 작은 변화**에 대한 비율이므로, 1 처럼 큰 변화에서는 근사값일 뿐입니다.

핵심 포인트

• ∂f/∂x = ∂f/∂q × ∂q/∂x : 중간 노드를 거쳐 미분값을 곱한다.
• 미리 구해 둔 로컬 그래디언트를 곱하기만 하면 된다.

확인 문제

Q1. x = 1, y = 2, z = 3 일 때 ∂f/∂x, ∂f/∂y, ∂f/∂z 를 구하시오. (3장 Q1 결과 이용)
Q2. ∂f/∂z = 3 은 무슨 뜻인지 해석하시오.

CHAPTER 5

로컬 그래디언트 × 글로벌 그래디언트

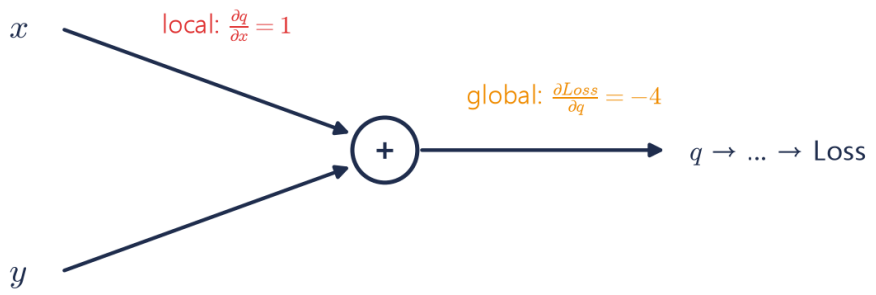
영상 구간 **07:43~10:43** 순전파 때 로컬 그래디언트 저장, 역전파 때 글로벌 그래디언트가 내려옴

학습 목표
- 로컬 그래디언트와 글로벌 그래디언트를 구분할 수 있다.
- 역전파의 핵심 공식 '글로벌 × 로컬'을 설명할 수 있다.

4장의 계산에서 등장한 값을 두 종류로 나눠 봅시다.

이름	뜻	언제 구하나	예제의 값
로컬 그래디언트	노드 출력 ÷ 그 노드의 직접 입력에 대한 미분	**순전파** 때 미리	∂q/∂x = 1, ∂q/∂y = 1
글로벌 그래디언트	최종 출력(손실)을 이 노드의 **출력**으로 미분한 값. 뒤에서부터 전달받아 내려온 값	**역전파** 때 뒤에서 전달	∂Loss/∂q = -4

이제 덧셈 노드 하나만 떼어서 보겠습니다. 이 노드 뒤에 수많은 계산이 더 있고 그 끝에 **손실(Loss)**이 있다고 해 봅시다. 뒤쪽 계산이 어떻게 되어 있든, 역전파가 이 노드까지 내려왔을 때 **글로벌 그래디언트 ∂Loss/∂q 하나만** 알면 됩니다.



$$\frac{\partial Loss}{\partial x} = \frac{\partial Loss}{\partial q} \cdot \frac{\partial q}{\partial x} = (-4)(1) = -4$$

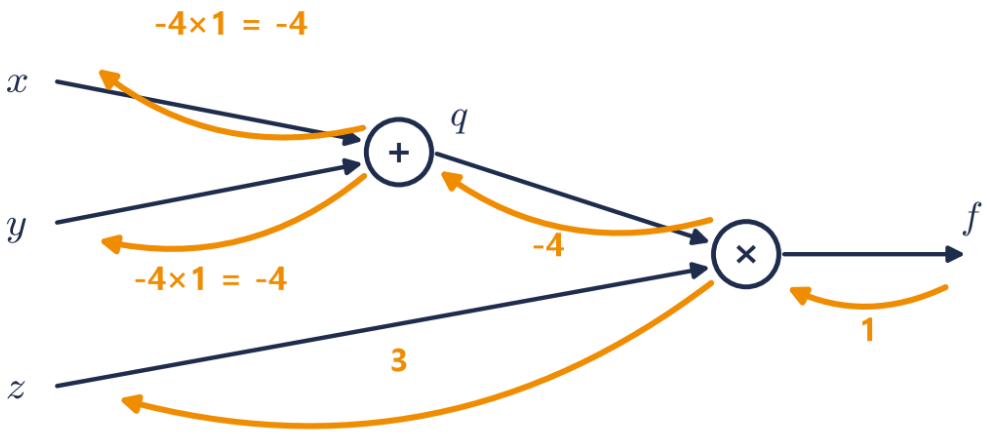
그림 5-1. 노드 하나의 역전파: 뒤에서 내려온 글로벌 그래디언트 × 내 로컬 그래디언트

역전파의 핵심 공식

$$\frac{\partial Loss}{\partial(\text{input})} = \frac{\partial Loss}{\partial(\text{output})} \times \frac{\partial(\text{output})}{\partial(\text{input})}$$

(내 입력에 대한 미분) = (뒤에서 내려온 글로벌 그래디언트) × (내 로컬 그래디언트)

이렇게 구한 값 ∂Loss/∂x 는 다시 x 앞쪽 노드에게는 **글로벌 그래디언트**가 되어 전달됩니다. 이것을 출력에서 입력까지 반복하는 것이 **역전파(Backward Pass, Backpropagation)**입니다. 맨 끝에서 시작할 때는 ∂f/∂f = 1 에서 출발합니다.



역전파 (Backward Pass) : 오른쪽 → 왼쪽으로 미분값을 전달하며 곱한다

그림 5-2. f = (x + y)z 의 전체 역전파. 주황색 값이 오른쪽에서 왼쪽으로 내려온다

이 방식의 장점은 **각 노드가 자기 일만 하면 된다**는 것입니다. 노드는 뒤쪽에 얼마나 복잡한 계산이 있는지 몰라도, 받은 값에 자기 로컬 그래디언트를 곱해 앞으로 넘기기만 하면 됩니다. 그래서 층이 아무리 깊어도 같은 규칙으로 계산할 수 있습니다.

주의

• 영상의 '글로벌 그래디언트'는 다른 책이나 강의(예: 스탠퍼드 CS231n)에서 **업스트림 그래디언트(upstream gradient)**라고 부르기도 합니다. 같은 개념입니다.

핵심 포인트

• 로컬 그래디언트: 순전파 때 저장. 글로벌 그래디언트: 역전파 때 뒤에서 내려옴.
• 내 입력의 미분 = 글로벌 × 로컬. 이 값이 다시 앞 노드의 글로벌이 된다.

확인 문제

Q1. 곱셈 노드 q = x · y 가 있다. x = 3, y = -1 이고 뒤에서 글로벌 그래디언트 ∂L/∂q = 2 가 내려왔다. ∂L/∂x 와 ∂L/∂y 를 구하시오.
Q2. 로컬 그래디언트를 순전파 때 미리 저장해 두면 좋은 점은 무엇인가?

CHAPTER 6

덧셈 · 곱셈 노드의 규칙

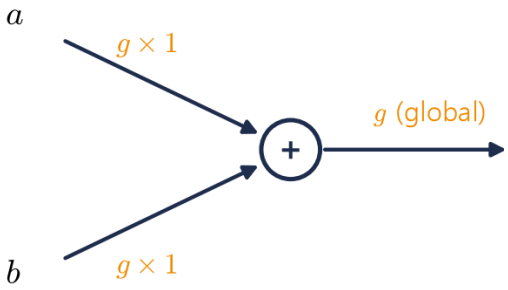
영상 구간 **10:43~11:25** 덧셈은 로컬 그래디언트 1, 곱셈은 서로 자리를 바꾼다

학습 목표

- 덧셈 노드와 곱셈 노드의 역전파 규칙을 외워서 바로 적용할 수 있다.

가장 많이 쓰이는 두 노드는 매번 미분하지 않아도 규칙만 기억하면 됩니다. 뒤에서 내려온 글로벌 그래디언트를 g 라고 하겠습니다.

덧셈 노드: 미분값을 그대로 나눠 준다



곱셈 노드: 상대 입력값을 곱한다 (자리 바꾸기)

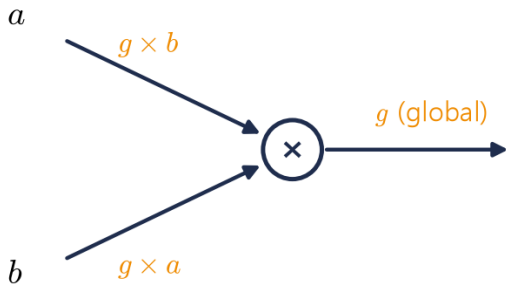


그림 6-1. 덧셈 노드와 곱셈 노드의 역전파 규칙

노드	순전파	로컬 그래디언트	역전파 (앞으로 보내는 값)	별명
덧셈	a + b	1, 1	g, g (그대로 복사)	분배기
곱셈	a × b	b, a (상대 값)	g × b, g × a (자리 바꾸기)	교환기

예제에 적용하면: 곱셈 노드 f = q · z 에 g = 1 이 들어오면 q 쪽으로는 1 × z = -4, z 쪽으로는 1 × q = 3 을 보냅니다. 덧셈 노드 q = x + y 는 받은 -4 를 x, y 에 그대로 -4, -4 로 나눠 줍니다. 5장 그림과 똑같죠.

핵심 포인트

- 덧셈 노드: 받은 그래디언트를 그대로 양쪽에 전달.
- 곱셈 노드: 받은 그래디언트 × 상대편 입력값. (그래서 순전파 때 입력값을 저장해야 한다)

확인 문제

Q1. f = (a + b) · c 에서 a = 1, b = 1, c = -2 이다. 순전파로 f 를 구하고, 규칙만 이용해 ∂f/∂a, ∂f/∂b, ∂f/∂c 를 구하시오.

Q2. 곱셈 노드의 역전파를 계산하려면 순전파 때 무엇을 저장해 두어야 하는가?

CHAPTER 7

정리와 코드 구현

영상 구간 **11:25~끝** 결론 3가지 (코드 구현은 영상 예고 → 이 교안에서 구현)

학습 목표

- 오차역전파의 핵심 세 가지를 정리할 수 있다.
- 덧셈·곱셈 노드를 클래스로 만들어 역전파를 구현할 수 있다.

영상의 결론

① 아무리 깊고 복잡한 층으로 구성되어 있어도 **연쇄법칙(Chain Rule)**을 활용해 미분값을 얻을 수 있다.
② **순전파** 때 **로컬 그래디언트를 미리 계산해 저장**해 둔다.
③ 저장해 둔 로컬 그래디언트와 **글로벌 그래디언트**를 **역전파** 때 곱해서 최종 미분값을 얻는다.

영상에서 "나중에 코드에서도 로컬 그래디언트를 저장하는 코드가 들어 있다"고 했습니다. 각 노드를 **forward()** 와 **backward()** 두 메서드를 가진 클래스로 만들어 봅시다. forward 는 값을 계산하면서 필요한 입력을 저장하고, backward 는 글로벌 그래디언트 dout 을 받아 6장 규칙대로 앞으로 보낼 값을 돌려줍니다.

실습 코드 7-1 - 덧셈 · 곱셈 노드 클래스로 역전파

```
class Add:
    def forward(self, a, b):
        return a + b
    def backward(self, dout):
        return dout * 1, dout * 1          # 덧셈: 그대로 나눠 준다

class Mul:
    def forward(self, a, b):
        self.a, self.b = a, b              # 순전파 때 입력을 저장 (= 로컬 그래디언트)
        return a * b
    def backward(self, dout):
        return dout * self.b, dout * self.a    # 곱셈: 서로 자리를 바꿔 곱한다

add, mul = Add(), Mul()

# 순전파
q = add.forward(x, y)
f = mul.forward(q, z)

# 역전파: 맨 끝에서 시작 (∂f/∂f = 1)
dq, dz = mul.backward(1)
dx, dy = add.backward(dq)
print("f =", f)
print("∂f/∂x =", dx, " ∂f/∂y =", dy, " ∂f/∂z =", dz)
```

실행 결과

```
f = -12
∂f/∂x = -4  ∂f/∂y = -4  ∂f/∂z = 3
```

손으로 계산한 4장 결과(-4, -4, 3)와 똑같이 나왔습니다. 역전파는 **마지막 노드부터 거꾸로** backward 를 호출한다는 점, 그리고 맨 처음에는 1 을 넣는다는 점을 확인하세요. PyTorch 나 TensorFlow 같은 딥러닝 프레임워크의 자동 미분(autograd)도 내부적으로 이와 같은 방식으로 동작합니다.

핵심 포인트

- 노드 = forward(값 계산 + 저장) + backward(글로벌 × 로컬).
- 역전파는 순전파의 역순으로 backward 를 호출, 시작값은 1.

확인 문제

Q1. Mul 클래스의 forward 에서 self.a, self.b 를 저장하지 않으면 어떤 문제가 생기는가?
Q2. 같은 Add, Mul 클래스로 f = x·y + z 를 계산하려 한다. x = -2, y = 5, z = -4 일 때 f 와 ∂f/∂x, ∂f/∂y, ∂f/∂z 를 구하시오.

CHAPTER 8

뉴런 하나에 적용하기 (보충)

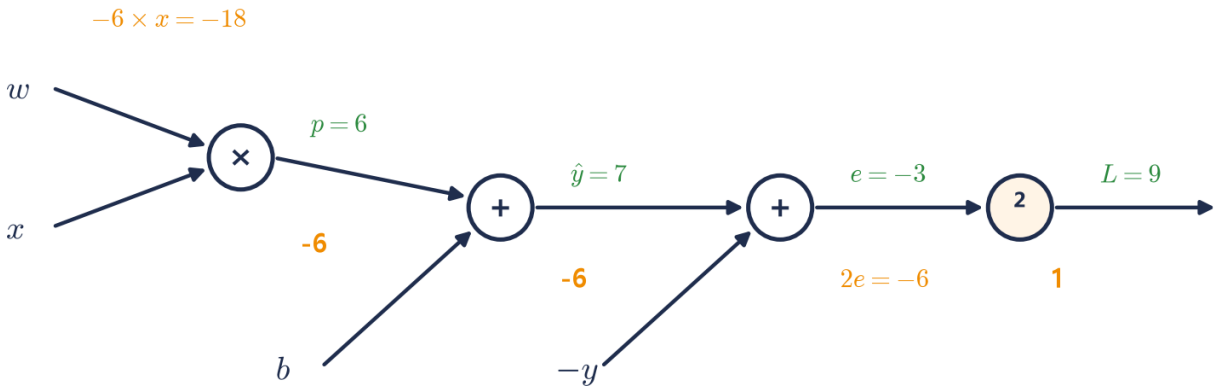
영상 구간 - 영상에 없는 보충 내용: 경사하강법 교안의 기울기 공식을 역전파로 다시 구하기

학습 목표

- 뉴런 하나와 제곱 오차를 계산 그래프로 그릴 수 있다.
- 역전파 결과가 손으로 유도한 공식과 같음을 확인한다.
- 역전파 + 경사하강법으로 선형 회귀를 학습시킬 수 있다.

8.1 데이터 1개로 손 계산

경사하강법 교안의 모델 ŷ = wx + b 와 제곱 오차 L = (ŷ - y)² 를 계산 그래프로 그리면 **곱셈 → 덧셈 → 덧셈(-y) → 제곱** 네 개의 노드가 됩니다. w = 2, x = 3, b = 1, y = 10 을 넣어 봅시다.



초록: 순전파 값 (w=2, x=3, b=1, y=10) 주황: 역전파로 내려온 미분값 (∂L/∂·)

그림 8-1. 뉴런 하나 + 제곱 오차의 계산 그래프. 초록은 순전파, 주황은 역전파

노드	순전파	로컬 그래디언트	역전파 (글로벌 × 로컬)
제곱 L = e²	L = (-3)² = 9	∂L/∂e = 2e = -6	1 × (-6) = **-6**
덧셈 e = ŷ + (-y)	e = 7 - 10 = -3	∂e/∂ŷ = 1	-6 × 1 = **-6**
덧셈 ŷ = p + b	ŷ = 6 + 1 = 7	∂ŷ/∂p = 1, ∂ŷ/∂b = 1	∂L/∂p = -6, **∂L/∂b = -6**
곱셈 p = w · x	p = 2 × 3 = 6	∂p/∂w = x = 3	**∂L/∂w = -6 × 3 = -18**

경사하강법 교안에서 손으로 유도한 공식은 ∂L/∂w = 2(ŷ - y)·x, ∂L/∂b = 2(ŷ - y) 였습니다. 값을 넣으면 2·(-3)·3 = -18, 2·(-3) = -6 으로 **역전파 결과와 정확히 같습니다**. 공식을 따로 유도하지 않아도, 노드 규칙만으로 같은 답이 나온 것입니다.

실습 코드 8-1 - 뉴런 하나의 역전파

```
class Square:
    def forward(self, a):
        self.a = a
        return a ** 2
    def backward(self, dout):
        return dout * 2 * self.a

# 뉴런 하나 + 제곱 오차:  L = (w*x + b - y)^2
w, b = 2.0, 1.0
x_in, y_true = 3.0, 10.0

mul, add1, add2, sq = Mul(), Add(), Add(), Square()
p     = mul.forward(w, x_in)          # p = w*x
y_hat = add1.forward(p, b)            # y_hat = p + b
e     = add2.forward(y_hat, -y_true)  # e = y_hat - y
L     = sq.forward(e)                 # L = e^2
print(f"순전파: p={p}, y_hat={y_hat}, e={e}, L={L}")

de = sq.backward(1)                   # ∂L/∂e = 2e
dy_hat, _ = add2.backward(de)
dp, db = add1.backward(dy_hat)
dw, dx = mul.backward(dp)
print(f"역전파: ∂L/∂e={de}, ∂L/∂y_hat={dy_hat}, ∂L/∂w={dw}, ∂L/∂b={db}")
print("공식 2(y_hat - y)*x =", 2 * (y_hat - y_true) * x_in, ", 2(y_hat - y) =", 2 * (y_hat - y_true))
```

실행 결과

```
순전파: p=6.0, y_hat=7.0, e=-3.0, L=9.0
역전파: ∂L/∂e=-6.0, ∂L/∂y_hat=-6.0, ∂L/∂w=-18.0, ∂L/∂b=-6.0
공식 2(y_hat - y)*x = -18.0 , 2(y_hat - y) = -6.0
```

8.2 데이터 30개로 학습하기

경사하강법 교안과 같은 데이터(y = 3x + 2 + 잡음, 30개)로 학습해 봅니다. MSE 의 평균(mean)도 하나의 노드로 만들면 됩니다. 평균 노드의 로컬 그래디언트는 각 입력마다 1/N 입니다. 그리고 w, b 는 30개 데이터 계산에 **공통으로** 쓰이므로, 30개에서 내려온 미분값을 **모두 더합니다**.

실습 코드 8-2 - 역전파 + 경사하강법으로 선형 회귀 학습

```
import numpy as np
rng = np.random.default_rng(42)
X = rng.uniform(0, 5, 30)
Y = 3 * X + 2 + rng.normal(0, 0.5, 30)

class Mean:
    def forward(self, a):
        self.n = len(a)
        return a.mean()
    def backward(self, dout):
        return dout * np.ones(self.n) / self.n

w, b, lr = 0.0, 0.0, 0.05
mul, add1, add2, sq, mean = Mul(), Add(), Add(), Square(), Mean()
for epoch in range(1, 501):
    # 순전파
    L = mean.forward(sq.forward(add2.forward(add1.forward(mul.forward(w, X), b), -Y)))
    # 역전파
    de = sq.backward(mean.backward(1))
    dy_hat, _ = add2.backward(de)
    dp, db_vec = add1.backward(dy_hat)
    dw_vec, _ = mul.backward(dp)
    dw, db = dw_vec.sum(), db_vec.sum()   # w, b 는 30개 데이터에 공통 -> 합친다
    w, b = w - lr * dw, b - lr * db
    if epoch in (1, 10, 100, 500):
        print(f"epoch {epoch:3d} | MSE = {L:8.4f} | w = {w:.4f}, b = {b:.4f}")
print("최소제곱법 :", np.round(np.polyfit(X, Y, 1), 4))
```

실행 결과

```
epoch   1 | MSE = 128.0895 | w = 3.5515, b = 1.0444
epoch  10 | MSE =   0.3519 | w = 3.2550, b = 1.1588
epoch 100 | MSE =   0.1615 | w = 3.0302, b = 1.9415
epoch 500 | MSE =   0.1560 | w = 2.9853, b = 2.0980
최소제곱법 : [2.9853 2.0981]
```

500 epoch 후 w = 2.9853, b = 2.0980 으로 최소제곱법의 답(2.9853, 2.0981)과 거의 같습니다. 경사하강법 교안의 결과와도 일치합니다. (이 코드는 정확한 기울기(2 포함)에 lr = 0.05 를 곱하므로, 영상식 업데이트에서 LR = 0.1 을 쓴 것과 같습니다.)

주의

- 하나의 값(w)이 여러 곳에서 쓰이면 역전파 때 각 경로에서 내려온 그래디언트를 **더해야** 합니다. 신경망에서 같은 가중치가 여러 데이터(배치)에 쓰일 때 항상 이 규칙이 적용됩니다.

핵심 포인트

- $\hat{y} = wx + b$ 와 MSE 도 계산 그래프. 역전파 결과 = 손으로 유도한 공식.
- 여러 곳에 쓰인 값의 그래디언트는 합친다.

확인 문제

Q1. w = 1, b = 0, x = 2, y = 5 일 때 그림 8-1 과 같은 방법으로 순전파(p, $\hat{y}$, e, L)와 역전파($\partial L/\partial e$, $\partial L/\partial w$, $\partial L/\partial b$)를 계산하시오.

Q2. 실습 코드 8-2 에서 dw_vec.sum() 처럼 합을 구하는 이유는 무엇인가?

APPENDIX A

확인 문제 정답 및 해설

1장

Q1. 오차역전파는 손실에 대한 각 가중치의 미분값(기울기)을 계산하고, 경사하강법은 그 기울기를 이용해 가중치를 업데이트한다.

Q2. 순전파는 입력 → 출력(앞 → 뒤), 역전파는 출력 → 입력(뒤 → 앞) 방향.

2장

Q1. q = 1 + 2 = 3, f = 3 × 3 = 9

Q2. 곱셈 노드 1개(a × b)와 덧셈 노드 1개(+ c), 총 2개.

3장

Q1. $\partial q/\partial x = 1$, $\partial q/\partial y = 1$, $\partial f/\partial q = z = 3$, $\partial f/\partial z = q = 3$

Q2. f = q·z 를 q 로 편미분하면 z 는 상수 취급되어 (q)′·z = 1·z = z 가 남기 때문이다.

4장

Q1. $\partial f/\partial x = \partial f/\partial q \cdot \partial q/\partial x = 3 \times 1 = 3$, $\partial f/\partial y = 3 \times 1 = 3$, $\partial f/\partial z = q = 3$

Q2. z 가 1 증가하면 f 가 3 만큼 증가한다는 뜻이다 (z = -3 이면 f = 3 × (-3) = -9 로 -12 에서 3 증가).

5장

Q1. $\partial L/\partial x = 2 \times y = 2 \times (-1) = -2$, $\partial L/\partial y = 2 \times x = 2 \times 3 = 6$

Q2. 역전파 때 다시 계산할 필요 없이 글로벌 그래디언트와 곱하기만 하면 되므로 계산이 빠르고 간단해진다. 각 노드가 뒤쪽 계산을 몰라도 된다.

6장

Q1. 순전파: a + b = 2, f = 2 × (-2) = -4. 곱셈 노드에 g = 1: c 쪽 = 1 × 2 = 2 → $\partial f/\partial c = 2$, (a + b) 쪽 = 1 × (-2) = -2. 덧셈 노드는 그대로 나눠 주므로 $\partial f/\partial a = -2$, $\partial f/\partial b = -2$.

Q2. 두 입력값 a, b (상대편 입력을 곱해야 하므로).

7장

Q1. backward 에서 곱해야 할 상대편 입력값을 알 수 없어 그래디언트를 계산할 수 없다 (AttributeError 발생).

Q2. m = mul.forward(x, y) = -10, f = add.forward(m, z) = -14. 역전파: dm, dz = add.backward(1) → 1, 1, dx, dy = mul.backward(dm) → 1 × y = 5, 1 × x = -2. 따라서 $\partial f/\partial x = 5$, $\partial f/\partial y = -2$, $\partial f/\partial z = 1$.

8장

Q1. 순전파: p = 1 × 2 = 2, $\hat{y}$ = 2 + 0 = 2, e = 2 - 5 = -3, L = 9. 역전파: $\partial L/\partial e = 2e = -6$, $\partial L/\partial \hat{y} = -6$, $\partial L/\partial b = -6$, $\partial L/\partial w = -6 \times x = -12$. (공식 $2(\hat{y} - y)x = 2\cdot(-3)\cdot 2 = -12$ 와 일치)

Q2. w 와 b 는 30개 데이터 모두의 계산에 같은 값으로 쓰인다. 하나의 값이 여러 경로로 손실에 영향을 주므로, 각 경로에서 내려온 그래디언트를 모두 더해야 전체 기울기가 된다.

APPENDIX
B

핵심 정리 한눈에 보기

구분	내용
예제 식	$f=(x+y)\,z, \quad q=x+y, \quad f=q\,z$
순전파	x = -2, y = 5, z = -4 → q = 3, f = -12
로컬 그래디언트	$\frac{\partial q}{\partial x}=1,\ \frac{\partial q}{\partial y}=1,\ \frac{\partial f}{\partial q}=z=-4,\ \frac{\partial f}{\partial z}=q=3$
연쇄법칙	$\frac{\partial f}{\partial x}=\frac{\partial f}{\partial q}\cdot\frac{\partial q}{\partial x}=(-4)(1)=-4$
역전파 공식	내 입력의 미분 = (뒤에서 내려온 글로벌 그래디언트) × (내 로컬 그래디언트)
덧셈 노드	받은 그래디언트 g 를 양쪽에 그대로 전달 (g, g)
곱셈 노드	g × 상대편 입력 (g·b, g·a) → 순전파 때 입력 저장 필요
시작값	맨 끝 출력에서 ∂f/∂f = 1 로 시작
여러 곳에 쓰인 값	각 경로의 그래디언트를 더한다

참고

원본 영상: 테디노트 TeddyNote, 「오차역전파 (Backpropagation)의 개념을 쉽게 이해해 봅시다」 (12분 5초, 2020.11.29)
https://www.youtube.com/watch?v=1Q_etC_GHHk
이 교안은 영상의 흐름을 따라 정리하고, 그림 · 실습 코드 · 확인 문제와 8장(보충)을 덧붙였습니다.